Manual de Usuario — Sistema de Recomendación de Restaurantes con Neo4j

1. Introducción

Este manual explica cómo poner en funcionamiento el sistema de recomendación de restaurantes: instalar y configurar Neo4j, cargar los datos de prueba y ejecutar las consultas Cypher de negocio y análisis de redes. Está orientado a un evaluador o usuario técnico que parta de una máquina limpia.

2. Requisitos previos

Componente	**Versión mínima**
Docker Engine	24+
Docker Compose	v2+
Python	3.8+ (solo para generar los CSV)
Navegador	Cualquiera (para Neo4j Browser)

En Linux, el usuario debe poder ejecutar `docker` sin `sudo`:

```
docker ps
```

Si falla, agregar el usuario al grupo `docker` y volver a iniciar sesión:

```
sudo usermod -aG docker $USER
```

3. Instalación y configuración de Neo4j

El motor se levanta con Docker Compose usando la definición del repositorio:

```
cd docker
docker compose up -d
```

Comprobar que el motor esté listo (puede tardar ~1 minuto al primer arranque):

```
cd ..
docker exec neo4j cypher-shell -u neo4j -p admin123 "RETURN 1 AS ok;"
```

Salida esperada:

```
ok
1
```

Acceso interactivo desde el navegador:

- **Neo4j Browser:** `http://localhost:7474`
- Usuario: `neo4j`
- Contraseña: `admin123`

3.1. Datos técnicos de conexión

Parámetro	**Valor**
Credenciales	`neo4j` / `admin123`
Bolt (drivers)	`bolt://localhost:7687`
HTTP / Browser	`http://localhost:7474`
Plugins	APOC habilitado
Importación CSV	`data/` montado como `/import` (solo lectura)

Si se desea cambiar la contraseña, editar `NEO4J_AUTH` en `docker/docker-compose.yml` antes del primer arranque y repetir el `compose up`.

4. Creación del esquema (constraints e índices)

Con el motor arriba, aplicar constraints y índices:

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
ddl/01_constraints.cypher
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
ddl/02_indexes.cypher
```

Verificar:

```
docker exec neo4j cypher-shell -u neo4j -p admin123 "SHOW CONSTRAINTS;"
docker exec neo4j cypher-shell -u neo4j -p admin123 "SHOW INDEXES;"
```

Deben aparecer 7 constraints y 10 índices adicionales, todos `ONLINE`.

5. Carga masiva de datos

5.1. Generar los CSV (opcional, ya incluidos en data/)

Los archivos CSV ya están versionados en `data/`. Para regenerarlos con la misma semilla (`20260001`) y resultados idénticos:

```
.venv/bin/python carga/gen_data.py
```

5.2. Cargar nodos y relaciones con LOAD CSV

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
carga/01_load_nodes.cypher
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
carga/02_load_rels.cypher
```

Los scripts usan `LOAD CSV WITH HEADERS`, `MERGE` (idempotente — puede reejecutarse sin duplicar) y lotes de `1000 ROWS`. Nota: en Neo4j 5.23+ la cláusula `USING PERIODIC COMMIT` fue eliminada; el equivalente es `CALL (row) { ... } IN TRANSACTIONS OF N ROWS`, que es lo que usan los scripts.

5.3. Verificar la carga

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
carga/verificar_carga.cypher
```

Conteos esperados: 500 usuarios, 200 restaurantes, 100 chefs, 50 platillos, 15 tipos de cocina y 15 562 relaciones (CALIFICÓ 2713 · VISITÓ 3492 · ES_AMIGO_DE 5000 · PERTENECE_A 379 · OFRECE 2090 · PREPARA 350 · TRABAJA_EN 277 · LE_GUSTA 1261), con cero violaciones de integridad.

5.4. Reiniciar la base (desarrollo)

Para vaciar todos los datos sin borrar constraints ni índices:

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
ddl/00_reset.cypher
```

6. Consultas de negocio (Q1–Q5)

Todas las consultas viven en `queries/`. Se ejecutan con `cypher-shell`. Cuando una consulta necesita parámetros, se pasan con `--param`.

Q1 — Recomendación social de restaurantes

Recomienda restaurantes calificados con ≥ 4 estrellas por amigos (directos e indirectos, hasta 3 grados de separación), excluyendo los que el usuario ya visitó. Primero por cantidad de amigos coincidentes y luego por calificación promedio.

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 \
  --param "email => 'paula.gonzalez.129@correo.com'" \
  < queries/q1_recomendacion_social.cypher
```

Cómo interpretar el resultado: `amigos_coincidentes` = cuántos amigos (hasta 3 grados) calificaron el restaurante con ≥ 4; `calificacion_prom` = promedio de esas calificaciones. El primer renglón es la recomendación más sólida. Cambiar el email por el de cualquier otro usuario del dataset.

Q2 — Chefs de alto impacto

Chefs que trabajan en restaurantes con calificación promedio > 4 y cuyos platillos se ofrecen en ≥ 3 restaurantes distintos.

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
queries/q2_chefs_alto_impacto.cypher
```

Cómo interpretar: `prom_restaurantes` = promedio de la calificación de los restaurantes donde trabaja; `restaurantes_platillos` = nº de restaurantes distintos que ofrecen algún platillo suyo. No requiere parámetros.

Q3 — Restaurantes infravalorados

Restaurantes con tasa de visitas con reserva por encima del promedio general, pero con calificación por debajo del promedio general de la plataforma.

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
queries/q3_infravalorados.cypher
```

Cómo interpretar: las columnas `tasa_global` y `rating_global` son las medias de toda la plataforma (≈ 0.500 y ≈ 3.93). Un restaurante listado concentra demanda (reservan) pero recibe menos estrellas que la media: candidato a revisar experiencia o expectativas.

Q4 — Precio declarado vs. consumo real

Compara el rango de precios declarado del restaurante (Económico 100 · Moderado 250 · Caro 500 · Premium 900) contra el consumo promedio real de sus visitas, ordenando por la diferencia.

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
queries/q4_precio_vs_consumo.cypher
```

Cómo interpretar: `diferencia` positiva significa que el ticket real supera lo esperado para su categoría (p. ej. un restaurante `Caro` que gasta como `Premium`, señal de rango mal declarado).

Q5 — Cocina declarada vs. realmente visitada

Detecta usuarios cuya cocina favorita declarada (`LE_GUSTA` con mayor `nivel_interes`) no coincide con la cocina que más visitan (`VISITÓ → PERTENECE_A`).

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
queries/q5_cocina_declarada_vs_visitada.cypher
```

Cómo interpretar: `discrepancia` es `true` cuando las cocinas difieren; se listan primero los usuarios con más visitas a la cocina real distinta. En el dataset, 461 de 500 usuarios presentan discrepancia.

7. Análisis de redes (N1–N2)

N1 — Grados de separación entre usuarios

Ruta más corta entre dos usuarios sobre la red de amistad (`shortestPath`).

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 \
  --param "a => 'marta.herrera.31@correo.com'" --param "b =>
'ana.rojas.73@correo.com'" \
  < queries/n1_grados_separacion.cypher
```

Cómo interpretar: `grados_de_separacion` es el número de saltos de amistad y `camino` la cadena de nombres intermedia. En el dataset el diámetro es 5 y la separación promedio 2.94 (red de "mundo pequeño").

N2 — Restaurantes altamente conectados

Ranking de restaurantes por número de chefs y platillos (conectividad = `num_chefs + num_platillos`).

```
docker exec -i neo4j cypher-shell -u neo4j -p admin123 <
queries/n2_restaurantes_conectados.cypher
```

Cómo interpretar: el primer renglón es el nodo gastronómico más central (más talento y oferta culinaria), útil para prioridades de promoción.

8. Resolución de problemas comunes

Problema	Causa frecuente	Solución
`Connection refused` en `bolt://localhost:7687` o `7474`	Contenedor detenido (p. ej. tras reiniciar el equipo).	`cd docker && docker compose up -d` y esperar a que el log muestre *"Bolt enabled"*.
`LOAD CSV` no encuentra el archivo (`file:///x.csv`)	El archivo no está en `data/` o la ruta es incorrecta.	Usar solo el nombre (sin prefijo), p. ej. `usuarios.csv`; los CSV deben estar en `data/` (montado en `/import`).
Error `Invalid input 'USING'` con `USING PERIODIC COMMIT`	Versión Neo4j 5.23+, cláusula eliminada.	Usar `CALL { ... } IN TRANSACTIONS OF N ROWS` (ya aplicado en los scripts).
`chown: operation not permitted` en la carpeta de datos del host	`/import` montado configurado como escritura.	La configuración monta `../data` como `:ro`; no modificar a `rw`.
Contraseña olvidada o los datos no persisten	Credenciales cambiadas o volumen recreado.	Revisar `NEO4J_AUTH` en `docker-compose.yml`; los datos viven en el volumen `neo4j_data`.
Puerto 7474/7687 ocupado	Otra instancia de Neo4j corriendo.	Mapear otros puertos en `docker-compose.yml` (p. ej. `7475:7474`).
La consola no muestra salida al usar `docker exec ... < archivo.cypher`	Falta `-i` para el stdin.	Usar siempre `docker exec -i neo4j cypher-shell ... < script.cypher`.
Reejecución de la carga duplica nodos	Se usó `CREATE` en vez de `MERGE`.	Los scripts usan `MERGE` y son idempotentes; si se usó `CREATE`, reiniciar con `ddl/00_reset.cypher`.
